Operating Systems Course Project

Project Proposal

Automated Unit Test Generator Agent

Project Title	Automated Unit Test Generator Agent
Course	Operating Systems
Project Type	Agentic AI Application Development
Duration	4 Weeks (approx. 10–15 hrs / week)
Student name	LIKAIQI24013579
Primary Language	Python 3.11+
Key Dependencies	OpenAI API · pytest · ast · subprocess
Document Version	v1.0 May 2026

1. Project Overview

1.1 Problem Statement

Writing unit tests is a critical practice for software quality assurance, yet it is one of the most commonly skipped steps — especially among student developers. Crafting comprehensive test suites that cover happy paths, edge cases, and exception scenarios is time-consuming and requires experience that beginners often lack. This project addresses that gap by building an autonomous agent that takes a Python source file as input, intelligently generates pytest test cases, executes them, analyses failures, and iteratively refines the tests until they pass — with no human intervention required.

1.2 Proposed Solution

The system uses a Large Language Model (LLM) as its reasoning core, combined with Python's built-in AST module for static analysis and pytest for dynamic test execution. The result is an agent capable of a closed "Observe → Reason → Act → Retry" loop: it parses function metadata, generates structured test code, runs the tests in an isolated subprocess, parses the failure output, and feeds that information back to the LLM for the next generation attempt.

1.3 Connection to Operating Systems Concepts

This project directly applies several core OS concepts taught in the course:

- Process Management: the agent spawns test execution as a child process using subprocess, providing process-level isolation and controlled resource usage.
- Inter-Process Communication (IPC): stdout and stderr are captured via pipes, and the structured output is parsed to understand the child process's outcome.
- File System Operations: generated test code is written to temporary files via tempfile, which the OS cleans up after execution.
- Resource Limits & Timeouts: per-test and global timeouts prevent runaway or infinite-looping generated code from blocking the agent — a direct parallel to OS-level process scheduling and protection.
- Signal Handling: the agent catches subprocess.TimeoutExpired exceptions, mirroring graceful degradation under SIGKILL conditions.

2. System Architecture

2.1 The Agentic Loop

The core of this system is an iterative Observe → Reason → Act loop — the defining characteristic that makes it an agent rather than a one-shot LLM call:

Step	Action	Description
① Parse	**AST static analysis**	Extract function name, argument types, return annotation, raised exceptions, and docstring using the ast module
② Reason	**LLM generates tests**	Construct a structured prompt from the metadata; call GPT-4o-mini to generate multi-scenario pytest test code
③ Act	**Subprocess execution**	Write generated code to a temp file; run pytest in an isolated child process with a timeout guard
④ Observe	**Parse results**	Use regex to extract each failing test name and error message from pytest's --tb=short output
⑤ Decide	**Stop or retry**	If all tests pass, output the file. Otherwise feed failures back to the LLM and trigger the next iteration.

2.2 Module Breakdown

The system comprises five decoupled modules, each with a single clear responsibility:

Module File	Tech Stack	Responsibility
`parser.py`	`ast, pathlib`	Extract function signatures, type annotations, and exception declarations via AST
`llm.py`	`openai SDK`	Wrap LLM API calls; strip markdown fences; control temperature for stable code output

prompts.py	f-string templates	Build the initial generation prompt and the failure-fix prompt with structured context
runner.py	subprocess, re, tempfile	Execute tests in a sandboxed child process; parse pytest output into structured results
orchestrator.py	All modules	Drive the main agent loop; manage state; enforce stop conditions

3. Technology Stack

3.1 Core Dependencies

Library	Version	Purpose
openai	>=1.30.0	Core reasoning engine — calls GPT-4o-mini to generate and fix test code
pytest	>=8.0.0	Test framework that executes generated tests and reports results
pytest-timeout	>=2.3.0	Enforces per-test time limits to prevent infinite loops from blocking the agent
pytest-cov	>=5.0.0	Measures line coverage of generated tests during Week 4 evaluation
typer	>=0.12.0	CLI framework supporting --explain, --output, and --max-attempts flags
coverage	>=7.5.0	Generates JSON coverage reports consumed by the evaluator
rich	>=13.7.0	Coloured terminal output for improved demo readability (optional)

3.2 Project File Structure

The complete directory layout (12 Python files, ~600 lines of core code):

```
test_gen_agent/ ├── main.py                          # CLI entry point (typer) ├──
requirements.txt           # Dependency manifest ├── .env.example                 #
Environment variable template ├── README.md                # Usage guide ├── agent/ │
├── __init__.py │     ├── orchestrator.py        # Agent main loop (core) │     ├──
parser.py                  # AST function metadata extraction │     ├── llm.py
# LLM API wrapper │     ├── prompts.py           # Prompt engineering templates │     └──
runner.py            # Sandboxed test execution ├── eval/ │     ├── __init__.py │     ├──
evaluator.py         # Evaluation framework │     └── fixtures/            # Benchmark
source files (10-15) └── tests/      └── test_parser.py        # Unit tests for the
parser module
```

4. Four-Week Implementation Plan

The project follows a "build the foundation first, then add intelligence" principle. Each week has a concrete deliverable that can be demonstrated independently.

Week	Theme	Tasks
Week 1	**AST Parsing & Test Execution**	• Set up project directory, virtual environment, and dependencies • Implement parser.py: extract function signatures, arg annotations, and raised exceptions via the ast module • Handle edge cases: *args, **kwargs, default arguments, class methods, async functions • Implement runner.py: subprocess sandbox execution + regex parsing of pytest output • Build a basic CLI in main.py (no LLM yet — wires up parser and runner only) • Write tests/test_parser.py to validate the parser module itself • Deliverable: run python main.py sample.py and see a printed function metadata report
Week 2	**LLM Integration & Prompt Engineering**	• Implement llm.py: wrap OpenAI API with error handling and markdown fence stripping • Implement prompts.py: design the structured initial prompt covering happy paths, boundary values, and exception scenarios • Iterate on the prompt with a single function type until LLM output quality is consistently good • Wire all modules together: Parse → Generate → Run into a single end-to-end flow (no retry yet) • Log every prompt and response to a file for debugging • Implement the --explain flag: generate a plain-English comment above each test explaining its intent • Deliverable: given a source file, produce test_sample.py that pytest can execute without errors
Week 3	**Agent Loop & Iterative Repair**	• Implement orchestrator.py: the full Observe → Reason → Act → Retry main loop • Implement build_fix_prompt: feed structured pytest failure info back to the LLM • Implement three stop conditions: all tests pass / max attempts reached / repetition detected (Jaccard similarity) • Intentionally introduce three types of bugs into benchmark functions and verify the agent detects and responds to failures • Compare pass rates at max_attempts=1 vs 2 vs 3 across different function complexities • Polish CLI flags: --max-attempts, --output, friendly error messages • Deliverable: a fully functional agent that demonstrates multi-round iterative repair
Week 4	**Evaluation & Final Report**	• Implement eval/evaluator.py: measure pass rate, line coverage, and bug-catch rate • Prepare 10–15 benchmark fixture files drawn from the Python standard library

		• Run the evaluator; produce a comparison table: single-attempt vs iterative repair pass rates • Run mutation testing: introduce deliberate bugs into source functions; measure how many are caught • Record a 3-minute demo video showing the full Observe→Reason→Act→Fix cycle • Write the final project report: background, architecture, experimental results, OS concept analysis, limitations, future work • Clean up code: remove debug logs, add docstrings, ensure reproducibility • Deliverable: eval_report.json + demo video + final report

4.1 Milestones at a Glance

Week	Milestone	Verifiable Deliverable
Week 1	**AST parsing pipeline ready**	python main.py sample.py prints a structured function metadata report
Week 2	**End-to-end single-pass generation**	Running the tool produces test_sample.py that pytest executes without errors
Week 3	**Full agentic retry loop working**	Feed a buggy function; observe pass rate climb across multiple agent iterations
Week 4	**Evaluation and demo complete**	eval_report.json + 3-min demo video + submitted final report

5. Evaluation Framework

In Week 4 the agent will be systematically benchmarked using three complementary metrics that together provide a complete picture of generated test quality:

Metric	Definition	Measurement Method
Pass Rate	Percentage of generated tests that execute successfully (pytest exit code 0)	pytest return code; count passed / total
Coverage Rate	Percentage of source code lines executed by the generated tests	pytest --cov with JSON report; read percent_covered field
Bug-Catch Rate (Mutation Testing)	Given an intentionally introduced bug, how often does the test suite detect it?	Manually mutate 10 bug points; measure test failure rate

Benchmark suite: 10–15 functions selected from Python's standard library (math, statistics, string modules), covering four categories: pure computation, exception-handling functions, string operations, and list processing.

Target thresholds (indicative): Pass Rate >= 80%, Coverage >= 70%, Bug-Catch Rate >= 60%. Actual values will be reported as-measured.

6. Technical Risks & Mitigations

Challenge	Risk	Mitigation Strategy
AST edge cases	lambda, nested functions, decorators may fail to parse cleanly	Week 1 scope limited to plain functions; complex syntax wrapped in try-except and logged
Unstable LLM output	Generated code may contain syntax errors or missing imports	runner.py first runs compile() for syntax validation; invalid code triggers retry without invoking pytest
Sandbox safety	Generated code might perform file I/O, network calls, or infinite loops	timeout=10 per test, timeout=60 globally; cwd and environment variables restricted
LLM output repetition	Agent gets stuck in a loop regenerating the same broken code	Jaccard similarity check on consecutive outputs; similarity > 0.95 forces termination
API cost	Multi-round iterations consume tokens quickly during development	Use gpt-4o-mini (low cost); cap max_tokens=2048; cache responses during local development

7. Project Highlights

7.1 Genuine Agentic Behaviour

This is not a simple LLM wrapper script. The system exhibits true autonomous behaviour: it perceives environmental state (pytest results), makes decisions (continue / stop / retry), takes actions (generate / repair code), and learns from failure. This mirrors the Observe–Reason–Act paradigm that underpins modern AI engineering practice.

7.2 Applied OS Concepts

The project grounds Operating Systems theory in working code. subprocess process creation and IPC, tempfile filesystem operations, and timeout-based resource limits are not theoretical — they are functional components that run on every invocation.

7.3 Quantifiable Evaluation

Three measurable metrics (pass rate, coverage, mutation catch rate) and a reproducible benchmark suite give the project scientific credibility. A results table in the final report transforms this from a demo into a verifiable experiment.

7.4 The --explain Flag

When --explain is enabled, the agent generates a plain-English comment above each test

explaining why that specific case matters. This single extra LLM call dramatically improves demo quality: reviewers immediately understand the agent's reasoning process without reading code.

8. Limitations & Future Work

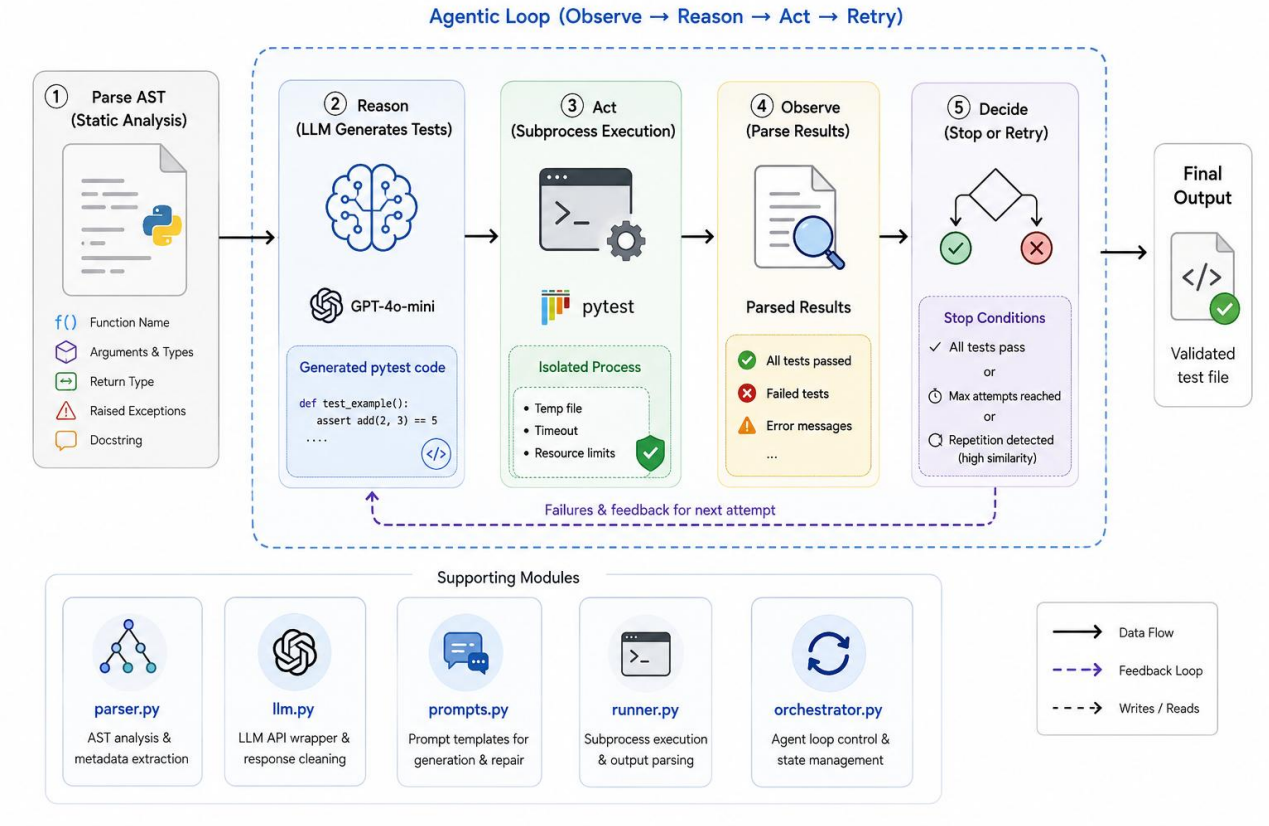
8.1 Current Limitations

- Supports pure functions and simple class methods only — functions with external I/O or database dependencies are out of scope.
- Generated tests do not use mocking, so functions that rely on external services are not well covered.
- AST analysis is static: runtime type information (e.g. dynamically typed arguments) cannot be captured.
- LLM reasoning quality degrades for algorithmically complex functions such as deep recursion or dynamic programming.

8.2 Potential Extensions

1. Mock support: integrate unittest.mock to handle functions with external dependencies.
2. Multi-language: extend the parsing layer to support JavaScript (via Esprima) or Java (via JavaParser).
3. CI/CD integration: package as a pre-commit hook or GitHub Actions step to auto-generate tests for every new commit.
4. Web UI: build a Gradio or Streamlit interface for non-CLI users.
5. Agent memory: store successful test patterns in a vector database to accelerate generation for similar functions.

9. System diagram



10. References

The following sources informed the design and implementation of this project:

- Python Documentation — ast module: https://docs.python.org/3/library/ast.html
- OpenAI Platform — Chat Completions API: https://platform.openai.com/docs/guides/chat
- pytest Documentation: https://docs.pytest.org/en/stable/
- Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629
- LangChain Agents Documentation: https://python.langchain.com/docs/modules/agents/
- Coverage.py Documentation: https://coverage.readthedocs.io/en/latest/

— End of Document —